Министерство образования Республики Беларусь

Учреждение образования
БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ

Факультет информационных технологий и управления

Кафедра информационных технологий автоматизированных систем

ОТЧЕТ
по лабораторной работе №5, часть 2
по дисциплине
«Современные системы программирования»

Выполнила: ст. гр. 320604
К. А. Хаджинова

Проверила: Ю. О. Герман

Минск 2026

СОДЕРЖАНИЕ

1 ЦЕЛЬ РАБОТЫ 3
2 ТЕОРЕТИЧЕСКИЕ СВЕДЕНИЯ 4
3 ВЫПОЛНЕНИЕ РАБОТЫ 5
3.1 Подготовка среды 5
3.2 Задание 1. Введение в параллельные циклы 5
3.3 Задание 2. *Parallel.ForEach* с коллекциями 6
3.4 Задание 3. *PLINQ* для анализа данных 8
3.5 Задание 4. Разделение данных и агрегация 10
3.6 Задание 5. Комплексная обработка изображений 11
3.7 Индивидуальное задание, вариант 2. Параллельная фильтрация коллекций 12
4 КОНТРОЛЬНЫЕ ВОПРОСЫ 16
4.1 Что такое *Task Parallel Library* (*TPL*) и каковы ее основные компоненты 16
4.2 В чем разница между *Parallel.For* и *Parallel.ForEach* 16
4.3 Чем *PLINQ* отличается от стандартного *LINQ* 16
4.4 Какие существуют стратегии разделения данных 16
4.5 Как выполняется агрегация результатов параллельных вычислений 16
4.6 Когда следует использовать параллельное программирование вместо последовательного 17
ЗАКЛЮЧЕНИЕ 18

1 ЦЕЛЬ РАБОТЫ

Изучить основы параллельного программирования в *C#*: библиотеку *Task Parallel Library* (*TPL*), параллельные циклы *Parallel.For* и *Parallel.ForEach*, *PLINQ* (*Parallel LINQ*), методы разделения данных и агрегации результатов.

2 ТЕОРЕТИЧЕСКИЕ СВЕДЕНИЯ

Task Parallel Library (*TPL*) – набор типов пространства имен *System.Threading.Tasks*, упрощающий параллелизм и асинхронность. *Task* представляет асинхронную операцию, *Task<TResult>* – операцию, возвращающую значение. *TPL* сама подбирает степень параллелизма под число доступных процессоров.

Параллельные циклы *Parallel.For* и *Parallel.ForEach* распределяют итерации по нескольким потокам, автоматически балансируют нагрузку, поддерживают локальные для потока данные, отмену операций и обработку исключений.

PLINQ – параллельная реализация *LINQ to Objects*. Вызов *AsParallel*() разбивает источник на части, и запрос выполняется на нескольких процессорах. Метод *AsOrdered*() сохраняет порядок исходной коллекции, *WithDegreeOfParallelism*(*n*) ограничивает число потоков.

Разделение данных (*partitioning*) – стратегия распределения данных между задачами: на диапазоны (массивы, списки), на небольшие блоки (динамическая балансировка нагрузки), по хешу (операции по ключу). Агрегация объединяет результаты параллельных вычислений. Эффективный способ – локальные для потока переменные: каждый поток накапливает результат у себя, а общая блокировка нужна один раз в конце.

3 ВЫПОЛНЕНИЕ РАБОТЫ

3.1 Подготовка среды

Приложение *Lab5* создано как консольный проект *.NET* с целевой платформой *net10.0* (*SDK* 10.0.401), замеры выполнены в режиме *Release* на процессоре *Intel Core i5-1035G1* (4 ядра, 8 логических процессоров). Задания запускаются аргументами *p2t1*, *p2t2*, *p2t3*, *p2t4*, *p2t5*, индивидуальное задание – аргументом *variant*.

В код методических указаний внесены минимальные правки. В задании 4 и 5 добавлено *using System.Collections.Concurrent*;, без которого класс *Partitioner* не компилируется. В задании 2 в интерактивном поиске убраны *Console.ReadKey*() в конце и бесконечный цикл при закрытом вводе: при *input* == *null* цикл завершается. Остальной код не изменен. Время каждого замера измерено однократно, поэтому цифры зависят от нагрузки компьютера.

3.2 Задание 1. Введение в параллельные циклы

Сумма *Math.Sqrt*(*Math.Log*(*data*[*i*] + 1)) по 10 000 000 элементов вычисляется тремя способами: последовательным циклом, *Parallel.For* с блокировкой на каждой итерации и *Parallel.For* с локальными переменными потока. Код параллельных вариантов приведен в листинге.

Листинг 3.1 – Параллельные циклы (задание 1)

```
// --- Параллельный цикл (Parallel.For) ---
sw.Restart();
double sumPar = 0;
object lockObj = new object();
Parallel.For(0, N, i =>
{
    double value = Math.Sqrt(Math.Log(data[i] + 1));
    lock (lockObj)
    {
        sumPar += value;
    }
});
sw.Stop();
Console.WriteLine($"Параллельно (с блокировкой): {sw.ElapsedMilliseconds} мс,
Результат: {sumPar:F2}");

// --- Параллельный цикл с локальными для потока переменными (оптимизированный) ---
sw.Restart();
double sumParLocal = 0;
Parallel.For(0, N,
    () => 0.0, // Локальный инициализатор
    (i, state, local) => local + Math.Sqrt(Math.Log(data[i] + 1)), // Тело
```

```
    local => { lock (lockObj) { sumParLocal += local; } } // Агрегатор
);
sw.Stop();
```

Результат показан на рисунке 3.1.

```
> dotnet run -c Release -- p2t1
=== ЗАДАНИЕ 1: ПАРАЛЛЕЛЬНЫЕ ЦИКЛЫ ===

Последовательно: 110 мс, Результат: 24215745.02
Параллельно (с блокировкой): 1239 мс, Результат: 24215745.02
Параллельно (локальные переменные): 42 мс, Результат: 24215745.02

=== Задание 1 Выполнено ===
```

Рисунок 3.1 – Результат задания 1

Последовательный цикл выполнился за 110 мс, вариант с блокировкой – за 1239 мс, вариант с локальными переменными – за 42 мс. Все три суммы одинаковы (24215745,02), то есть результат корректен.

Данные по теме «причины замедления версии с блокировкой» приведены в таблице 3.1.

Таблица 3.1 – Причины замедления версии с блокировкой

Фактор	Влияние
Блокировка на каждой итерации	Оператор `*lock*` вызывается 10 миллионов раз, накладные расходы превышают полезную работу
Конкуренция за ресурс	Все потоки борются за доступ к одному объекту блокировки
Переключение контекста	Потоки часто блокируются и возобновляются
Кэш-промахи	Общие данные синхронизации сбрасываются из кэшей процессоров

Версия с блокировкой оказалась в 11 раз медленнее последовательной, а версия с локальными переменными – в 2,6 раза быстрее последовательной и в 29 раз быстрее версии с блокировкой, так как общая блокировка берется только один раз на поток.

3.3 Задание 2. *Parallel.ForEach* с коллекциями

Задание содержит пять частей: параллельную трансформацию массива из 10 000 000 чисел, фильтрацию через *Parallel.ForEach* с локальными списками, поиск с отменой через *CancellationToken*, сравнение блокировки и локальных переменных и интерактивный поиск числа. Код фильтрации и поиска с отменой приведен в листинге.

Листинг 3.2 – Фильтрация и поиск с отменой (задание 2)

```
// 2. Фильтрация (Parallel.ForEach с локальными переменными)
Console.WriteLine("--- Фильтрация (>500 и кратные 7) ---");
List<int> filtered = new List<int>();
object listLock = new object();
var ranges = Enumerable.Range(0, N).Select(i => numbers[i]);
sw.Restart();
Parallel.ForEach(ranges,
    () => new List<int>(),
    (val, state, local) =>
    {
        if (val > 500 && val % 7 == 0) local.Add(val);
        return local;
    },
    local => { lock (listLock) filtered.AddRange(local); }
);
sw.Stop();
Console.WriteLine($"Найдено: {filtered.Count}, Время: {sw.ElapsedMilliseconds} мс");
Console.WriteLine($"Примеры: {string.Join(", ", filtered.Take(5))}\n");

// 3. Поиск с отменой
Console.WriteLine("--- Поиск с отменой ---");
int[] targets = { 777, 999, 123 };
bool[] found = new bool[targets.Length];
int[] positions = new int[targets.Length];
CancellationTokenSource cts = new CancellationTokenSource();
sw.Restart();
try
{
    Parallel.For(0, N, new ParallelOptions { CancellationToken = cts.Token }, (i, state) =>
    {
        for (int j = 0; j < targets.Length; j++)
        {
            if (!found[j] && numbers[i] == targets[j])
            {
                lock (found)
                {
                    if (!found[j])
                    {
                        found[j] = true;
                        positions[j] = i;
                        Console.WriteLine($"✅ Найдено {targets[j]} на позиции {i}");
                        if (found.All(f => f)) { cts.Cancel(); state.Break(); return; }
                    }
                }
            }
        }
    });
}
catch (OperationCanceledException) { Console.WriteLine("✅ Все найдено!"); }
sw.Stop();
Console.WriteLine($"Время поиска: {sw.ElapsedMilliseconds} мс\n");
```

Результат показан на рисунке 3.2.

```
> dotnet run -c Release -- p2t2
=== ЗАДАНИЕ 2: ПАРАЛЛЕЛЬНАЯ ОБРАБОТКА ДАННЫХ ===

--- Трансформация данных ---
Время: 96 мс
Пример: 27.02, 5.10, 379.00

--- Фильтрация (>500 и кратные 7) ---
Найдено: 710300, Время: 369 мс
Примеры: 595, 966, 854, 917, 910

--- Поиск с отменой ---
 Найдено 999 на позиции 3750123
 Найдено 123 на позиции 7500243
 Найдено 777 на позиции 3750766
 Все найдено!
Время поиска: 14 мс

--- Сравнение: блокировка vs локальные ---
С блокировкой: 965 мс
С локальными: 21 мс
Результаты: 3,331,288,557,750 = 3,331,288,557,750

=== Интерактивный поиск ===
Введите число (или 'exit'):
> 777
 Найдено на позиции 769
> 5000
 Не найдено
> exit

=== Задание 2 Выполнено ===
```

Рисунок 3.2 – Результат задания 2

Трансформация выполнена за 96 мс. Фильтрация нашла 710 300 чисел, которые больше 500 и кратны 7, за 369 мс. Поиск с отменой нашел все три значения (999, 123, 777) за 14 мс и остановил цикл. Сравнение сумм: с блокировкой 965 мс, с локальными переменными 21 мс (в 46 раз быстрее), результаты совпали: 3 331 288 557 750. В интерактивном режиме число 777 найдено на позиции 769, а число 5000, которого нет в диапазоне от 1 до 999, не найдено.

Позиции найденных значений в поиске с отменой отличаются от запуска к запуску, так как итерации выполняются в произвольном порядке разными потоками, а числа генерируются случайно. Метод *state.Break*() гарантирует выполнение только итераций с меньшими индексами, поэтому интерактивный поиск возвращает не обязательно первую позицию.

3.4 Задание 3. *PLINQ* для анализа данных

Среднее значение корней чисел, превышающих 500, вычисляется для 10 000 000 вещественных чисел последовательным *LINQ*, *PLINQ* по

умолчанию, *PLINQ* со степенью параллелизма 4 и упорядоченным *PLINQ* с *Take*(5). Код приведен в листинге.

Листинг 3.3 – Запросы *LINQ* и *PLINQ* (задание 3)

```
// --- Последовательный LINQ ---
Stopwatch sw = Stopwatch.StartNew();
var seqResult = data
    .Where(x => x > 500)
    .Select(x => Math.Sqrt(x))
    .Average();
sw.Stop();
Console.WriteLine($"Последовательный LINQ: {sw.ElapsedMilliseconds} мс, Результат:
{seqResult:F4}");

// --- PLINQ (базовый) ---
sw.Restart();
var plinqResult = data
    .AsParallel()
    .Where(x => x > 500)
    .Select(x => Math.Sqrt(x))
    .Average();
sw.Stop();
Console.WriteLine($"PLINQ (по умолчанию): {sw.ElapsedMilliseconds} мс, Результат:
{plinqResult:F4}");

// --- PLINQ со степенью параллелизма ---
sw.Restart();
var plinqResult2 = data
    .AsParallel()
    .WithDegreeOfParallelism(4)
    .Where(x => x > 500)
    .Select(x => Math.Sqrt(x))
    .Average();
sw.Stop();
Console.WriteLine($"PLINQ (степень=4): {sw.ElapsedMilliseconds} мс, Результат:
{plinqResult2:F4}");

// --- PLINQ с сохранением порядка ---
```

Результат показан на рисунке 3.3.

```
> dotnet run -c Release -- p2t3
=== ЗАДАНИЕ 3: PLINQ АНАЛИЗ ДАННЫХ ===

Последовательный LINQ: 153 мс, Результат: 27.2567
PLINQ (по умолчанию): 273 мс, Результат: 27.2567
PLINQ (степень=4): 228 мс, Результат: 27.2567
PLINQ (упорядоченный): 25 мс
Первые 5 результатов: 1099.9864341435382, 1948.2650733553835, 1238.9977581458809, 1728.8603282934178
, 1292.8798737091852

=== Задание 3 Выполнено ===
```

Рисунок 3.3 – Результат задания 3

Последовательный *LINQ* выполнился за 153 мс, *PLINQ* по умолчанию – за 273 мс, *PLINQ* со степенью 4 – за 228 мс. Средние значения у всех трех вариантов одинаковы (27,2567). Упорядоченный запрос с *Take*(5) занял 25 мс, потому что остановился после получения первых пяти элементов.

В этом задании *PLINQ* оказался медленнее последовательного *LINQ*. Операции над одним числом (сравнение и квадратный корень) дешевы, поэтому затраты на разбиение данных, запуск потоков и объединение результатов превышают выигрыш, а запрос к *List<double>* упирается в скорость чтения памяти. Параллелизм дает выигрыш на более тяжелых операциях, как показано в индивидуальном задании.

3.5 Задание 4. Разделение данных и агрегация

Сумма 10 000 000 чисел вычисляется тремя способами: *Parallel.For* с *Interlocked.Add* на каждой итерации (разделение на диапазоны по умолчанию), *Partitioner.Create*(0, *N*, 100000) с локальной суммой на блок и *Parallel.For* с локальной для потока агрегацией. Код приведен в листинге.

Листинг 3.4 – Стратегии разделения данных (задание 4)

```
// --- Пользовательское разделение с Partitioner ---
sw.Restart();
long sumCustom = 0;
var partitioner = Partitioner.Create(0, N, 100000);
Parallel.ForEach(partitioner, range =>
{
    long localSum = 0;
    for (int i = range.Item1; i < range.Item2; i++)
    {
        localSum += data[i];
    }
    Interlocked.Add(ref sumCustom, localSum);
});
sw.Stop();
Console.WriteLine($"Пользовательское разделение: {sw.ElapsedMilliseconds} мс, Сумма:
{sumCustom}");

// --- Локальная для потока агрегация ---
sw.Restart();
long sumLocal = 0;
Parallel.For(0, N,
    () => 0L,
    (i, state, local) => local + data[i],
    local => Interlocked.Add(ref sumLocal, local)
);
sw.Stop();
Console.WriteLine($"Локальная агрегация: {sw.ElapsedMilliseconds} мс, Сумма:
{sumLocal}");
```

Результат показан на рисунке 3.4.

```
> dotnet run -c Release -- p2t4
=== ЗАДАНИЕ 4: РАЗДЕЛЕНИЕ ДАННЫХ ===

Разделение на диапазоны: 237 мс, Сумма: 499978259
Пользовательское разделение: 8 мс, Сумма: 499978259
Локальная агрегация: 19 мс, Сумма: 499978259

=== Задание 4 Выполнено ===
```

Рисунок 3.4 – Результат задания 4

Разделение на диапазоны с атомарным сложением заняло 237 мс, пользовательское разделение блоками по 100 000 элементов – 8 мс, локальная агрегация – 19 мс. Суммы во всех трех случаях равны 499 978 259.

Самый медленный вариант обращается к общей переменной на каждой итерации, а в двух других синхронизация выполняется один раз на блок или поток. Блочный *Partitioner* оказался самым быстрым, так как внутри блока цикл выполняется без накладных расходов делегата на каждый элемент.

3.6 Задание 5. Комплексная обработка изображений

Изображение моделируется двумерным массивом 2000 на 2000 пикселей. К каждому пикселю применяется функция *ProcessPixel*. Сравниваются последовательная обработка, *Parallel.For* по строкам, *Partitioner* блоками по 100 строк и *PLINQ*. Код приведен в листинге.

Листинг 3.5 – Параллельная обработка изображения (задание 5)

```
// --- Параллельная обработка (по строкам) ---
sw.Restart();
Parallel.For(0, height, y =>
{
    for (int x = 0; x < width; x++)
    {
        result[y, x] = ProcessPixel(image[y, x]);
    }
});
sw.Stop();
Console.WriteLine($"Параллельно (по строкам): {sw.ElapsedMilliseconds} мс");

// --- Параллельная обработка с разделением ---
sw.Restart();
var partitioner = Partitioner.Create(0, height, 100);
Parallel.ForEach(partitioner, range =>
{
    for (int y = range.Item1; y < range.Item2; y++)
```

```
    {
        for (int x = 0; x < width; x++)
        {
            result[y, x] = ProcessPixel(image[y, x]);
        }
    }
});
sw.Stop();
Console.WriteLine($"Параллельно (с разделением): {sw.ElapsedMilliseconds} мс");

// --- Обработка с помощью PLINQ ---
sw.Restart();
var pixels = Enumerable.Range(0, height * width)
    .AsParallel()
    .Select(i => ProcessPixel(image[i / width, i % width]))
    .ToArray();
sw.Stop();
Console.WriteLine($"PLINQ: {sw.ElapsedMilliseconds} мс");
```

Результат показан на рисунке 3.5.

```
> dotnet run -c Release -- p2t5
=== ЗАДАНИЕ 5: ПАРАЛЛЕЛЬНАЯ ОБРАБОТКА ИЗОБРАЖЕНИЙ ===

Последовательно: 83 мс
Параллельно (по строкам): 36 мс
Параллельно (с разделением): 20 мс
PLINQ: 104 мс

=== Задание 5 Выполнено ===
```

Рисунок 3.5 – Результат задания 5

Последовательная обработка заняла 83 мс, *Parallel.For* по строкам – 36 мс, *Partitioner* – 20 мс, *PLINQ* – 104 мс. Лучший результат у блочного разделения (в 4,2 раза быстрее последовательной обработки). *PLINQ* проиграл, так как создает промежуточный массив и обращается к двумерному массиву через вычисление индексов *i* / *width* и *i* % *width*.

3.7 Индивидуальное задание, вариант 2. Параллельная фильтрация коллекций

По варианту 2 требуется реализовать параллельную фильтрацию коллекций с помощью *PLINQ* и пользовательских условий. Условия оформлены как делегаты *Func<int, bool>*, которые передаются в один и тот же метод фильтрации: «значение больше 500 000 и кратно 7» (дешевое условие) и «значение простое» (затратное условие, проверка делением). Фильтруется коллекция из 2 000 000 чисел четырьмя способами: *LINQ*,

PLINQ, *PLINQ* с *AsOrdered*() и *Parallel.ForEach* с локальными списками. Код приведен в листинге.

Листинг 3.6 – Фильтрация с пользовательскими условиями (вариант 2)

```
// Пользовательские условия фильтрации
static readonly (string Name, Func<int, bool> Condition)[] Conditions =
{
    ("значение > 500000 и кратно 7", x => x > 500000 && x % 7 == 0),
    ("значение простое", IsPrime),
};

public static void Run()
{
    Console.WriteLine("=== ВАРИАНТ 2: ПАРАЛЛЕЛЬНАЯ ФИЛЬТРАЦИЯ
КОЛЛЕКЦИЙ ===\n");

    const int N = 2000000;
    Random rand = new Random(42);
    List<int> data = Enumerable.Range(0, N).Select(i => rand.Next(1, 1000000)).ToList();
    Console.WriteLine($"Коллекция: {N} чисел от 1 до 999999\n");

    foreach (var (name, condition) in Conditions)
    {
        Console.WriteLine($"--- Условие: {name} ---");
        Filter(data, condition);
        Console.WriteLine();
    }

    Console.WriteLine("=== Вариант 2 Выполнен ===");
}

static void Filter(List<int> data, Func<int, bool> condition)
{
    // Последовательный LINQ
    Stopwatch sw = Stopwatch.StartNew();
    List<int> seq = data.Where(condition).ToList();
    sw.Stop();
    Console.WriteLine($"LINQ:                {sw.ElapsedMilliseconds,5} мс, найдено
{seq.Count}");

    // PLINQ: порядок результатов не гарантируется
    sw.Restart();
    List<int> plinq = data.AsParallel().Where(condition).ToList();
    sw.Stop();
    Console.WriteLine($"PLINQ:               {sw.ElapsedMilliseconds,5} мс, найдено
{plinq.Count}");

    // PLINQ с сохранением порядка исходной коллекции
    sw.Restart();
    List<int> ordered = data.AsParallel().AsOrdered().Where(condition).ToList();
    sw.Stop();
```

```
        Console.WriteLine($"PLINQ (AsOrdered):      {sw.ElapsedMilliseconds,5} мс, найдено
{ordered.Count}");

        // Parallel.ForEach с локальными списками потоков
        sw.Restart();
        List<int> merged = new List<int>();
        object mergeLock = new object();
        Parallel.ForEach(data,
            () => new List<int>(),
            (value, state, local) => { if (condition(value)) local.Add(value); return local; },
            local => { lock (mergeLock) merged.AddRange(local); });
        sw.Stop();
        Console.WriteLine($"Parallel.ForEach:       {sw.ElapsedMilliseconds,5} мс, найдено
{merged.Count}");

        Console.WriteLine($"Результаты LINQ и PLINQ (AsOrdered) совпадают:
{seq.SequenceEqual(ordered)}");
        Console.WriteLine($"Первые 5: {string.Join(", ", ordered.Take(5))}");
    }
}
```

Результат показан на рисунке 3.6.

```
> dotnet run -c Release -- variant
=== ВАРИАНТ 2: ПАРАЛЛЕЛЬНАЯ ФИЛЬТРАЦИЯ КОЛЛЕКЦИЙ ===

Коллекция: 2000000 чисел от 1 до 999999

--- Условие: значение > 500000 и кратно 7 ---
LINQ:                        24 мс, найдено 143321
PLINQ:                       36 мс, найдено 143321
PLINQ (AsOrdered):           33 мс, найдено 143321
Parallel.ForEach:            25 мс, найдено 143321
Результаты LINQ и PLINQ (AsOrdered) совпадают: True
Первые 5: 761250, 705152, 821065, 654143, 913514

--- Условие: значение простое ---
LINQ:                       441 мс, найдено 156168
PLINQ:                       96 мс, найдено 156168
PLINQ (AsOrdered):          105 мс, найдено 156168
Parallel.ForEach:            91 мс, найдено 156168
Результаты LINQ и PLINQ (AsOrdered) совпадают: True
Первые 5: 173651, 234589, 186379, 149119, 289607

=== Вариант 2 Выполнен ===
```

Рисунок 3.6 – Результат индивидуального задания

Дешевое условие: *LINQ* – 24 мс, *PLINQ* – 36 мс, *PLINQ* с сохранением порядка – 33 мс, *Parallel.ForEach* – 25 мс, найдено 143 321 число. Затратное условие (простые числа): *LINQ* – 441 мс, *PLINQ* – 96 мс, *PLINQ* с сохранением порядка – 105 мс, *Parallel.ForEach* – 91 мс, найдено 156 168 чисел. Результаты *LINQ* и упорядоченного *PLINQ* совпали полностью.

Эффект от распараллеливания зависит от стоимости условия. Для дешевого условия параллельные варианты не быстрее последовательного, а для затратного *PLINQ* ускоряет фильтрацию в 4,6 раза (441 мс против 96 мс). Включение *AsOrdered*() сохраняет порядок исходной коллекции ценой небольшого замедления (105 мс против 96 мс).

4 КОНТРОЛЬНЫЕ ВОПРОСЫ

4.1 Что такое *Task Parallel Library* (*TPL*) и каковы ее основные компоненты

TPL – библиотека пространства имен *System.Threading.Tasks* для параллельного и асинхронного программирования. Основные компоненты: *Task* и *Task<TResult>* (асинхронные операции), *Parallel.For*, *Parallel.ForEach* и *Parallel.Invoke* (параллельные циклы и вызовы), *CancellationToken* (отмена), *Partitioner* (разделение данных), а также *PLINQ*.

4.2 В чем разница между *Parallel.For* и *Parallel.ForEach*

Parallel.For распараллеливает цикл по диапазону целых индексов и подходит для массивов и списков. *Parallel.ForEach* перебирает произвольную коллекцию *IEnumerable* и подходит там, где индексов нет, а также для работы с *Partitioner*.

4.3 Чем *PLINQ* отличается от стандартного *LINQ*

Стандартный *LINQ* выполняет запрос в одном потоке. *PLINQ* после *AsParallel*() разбивает источник на части и выполняет запрос на нескольких процессорах, а затем объединяет результаты. Порядок результатов при этом не гарантируется, если не вызвать *AsOrdered*().

4.4 Какие существуют стратегии разделения данных

Разделение на диапазоны (непрерывные части, для массивов и списков), блочное разделение (небольшие блоки с динамической балансировкой нагрузки) и хеш-разделение (распределение по хешу ключа, для операций по ключу).

4.5 Как выполняется агрегация результатов параллельных вычислений

Каждый поток накапливает промежуточный результат в локальной переменной, а в конце результаты потоков объединяются в общий: через *Interlocked.Add*, под блокировкой *lock* или методами *PLINQ* (*Sum*, *Average*, *Aggregate*). Такой подход избегает блокировки на каждой итерации.

4.6 Когда следует использовать параллельное программирование вместо последовательного

Когда объем вычислений на элемент достаточно велик по сравнению с накладными расходами на потоки (большие массивы, тяжелые вычисления, обработка изображений), итерации независимы и есть несколько ядер. Для коротких операций над малыми данными последовательный код быстрее, что показали задание 3 и дешевое условие в индивидуальном задании.

ЗАКЛЮЧЕНИЕ

Изучены средства параллельного программирования *C#*: параллельные циклы *Parallel.For* и *Parallel.ForEach*, отмена операций, *PLINQ*, разделение данных *Partitioner* и локальная агрегация. Измерения показали, что блокировка на каждой итерации замедляет параллельный цикл в 11 раз по сравнению с последовательным, а локальные для потока переменные ускоряют его в 2,6 раза. *PLINQ* выигрывает только на затратных операциях: при проверке чисел на простоту ускорение составило 4,6 раза, а для дешевых операций он оказался медленнее обычного *LINQ*. Выполнено индивидуальное задание варианта 2 – параллельная фильтрация коллекции с пользовательскими условиями. Цель работы достигнута.